

Documento de investigación y Diseño

Presentación 1

Arquitectura de Software

- Tomas Silva
- Alexander Aponte
- Juan Pabón Vargas

Investigación de Estilo Arquitectónico.....	2
Layered Architecture.....	2
Definición clara (qué es y qué no es).....	2
Clasificación del estilo.....	3
Características principales.....	3
Historia y evolución.....	3
Ventajas y desventajas.....	4
Problemas comunes que se presentan y patrones.....	4
Identificar patrones aplicables y cuando usarlos.....	5
Casos de uso (cuándo usarlo y cuándo no).....	6
Casos de aplicación (ejemplos reales en la industria).....	6
Frontend - Angular/JS.....	7
Definición clara (qué es y qué no es).....	7
Características principales.....	7
Historia y evolución.....	8
Ventajas y desventajas.....	8
Casos de uso (cuándo usarlo y cuándo no).....	8
Casos de aplicación (ejemplos reales en la industria).....	9
Relación entre el estilo y las tecnologías seleccionadas.....	9
Que tan común es el stack designado (relación entre tecnologías).....	9
Backend - Fast API/Python.....	10
Definición clara (qué es y qué no es).....	10
Características principales.....	10
Historia y evolución.....	11
Ventajas y desventajas.....	11
Casos de uso (cuándo usarlo y cuándo no).....	11
Casos de aplicación (ejemplos reales en la industria).....	12
Relación entre el estilo y las tecnologías seleccionadas.....	12
Que tan común es el stack designado (relación entre tecnologías).....	12
Base de Datos - MongoDB/Document.....	12
Definición clara (qué es y qué no es).....	12
Características principales.....	13
Historia y evolución.....	13
Ventajas y desventajas.....	13
Casos de uso (cuándo usarlo y cuándo no).....	14

Casos de aplicación (ejemplos reales en la industria).....	14
Relación entre el estilo y las tecnologías seleccionadas.....	14
Integración - Rest, JSON, API.....	15
Definición clara (qué es y qué no es).....	15
Características principales.....	15
Historia y evolución.....	15
Ventajas y desventajas.....	16
Casos de uso (cuándo usarlo y cuándo no).....	16
Casos de aplicación (ejemplos reales en la industria).....	17
Relación entre el estilo y las tecnologías seleccionadas.....	17
Análisis Arquitectónico.....	18
Matriz de Atributos de Calidad vs. Estilo.....	18
Matriz de Principios vs. Estilo.....	19
Matriz de Tácticas vs. Estilo y Stack (insumo para ADRs).....	20
Matriz de Mercado Laboral vs. Estilo y Stack.....	22
Diseño.....	24
Entidades de negocio interrelacionadas.....	24
HLD (High-Level Design).....	25
Diagrama de Contexto (C4 - Nivel 1).....	25
Diagrama de Contenedores (C4 - Nivel 2).....	25
Diagrama Dinámico — Flujo principal: "Rentar un disco".....	25
Diagrama de Despliegue.....	25
Lecciones Aprendidas.....	25
Fuentes.....	25

Investigación de Estilo Arquitectónico

Layered Architecture

N-Tiers, Local o Distribuido, Capas Cerradas vs. Abiertas.

Definición clara (qué es y qué no es)

Es:

- Un patrón de diseño que organiza el código de una aplicación en capas con roles definidos.

No es:

- Un sinónimo de microservicios, ya que no se están dividiendo servicios independientes en la red, sino que en su lugar, las tareas se producen en capas en un mismo bloque.
- No es código acoplado que permite saltarse niveles (a menos que se defina con sus reglas de capas abiertas y cerradas) ni permite ciertas acciones en capas donde no corresponde.

- No define un número específico de capas

Clasificación del estilo.

Se considera como un estilo arquitectónico **Estructural** o de organización basado en capas donde se toma en cuenta:

N-Tiers

Organiza el sistema en un número determinado de capas o niveles, donde cada una tiene una responsabilidad específica. La cantidad de niveles depende de las necesidades de la aplicación. Se usa principalmente cuando se necesita separar responsabilidades como presentación, lógica de negocio y persistencia en diferentes niveles.

Local o Distribuido

Local: todas las capas se ejecutan dentro de un mismo entorno o aplicación, sin estar separadas físicamente entre diferentes máquinas o servidores.

Distribuido: las capas se ejecutan en diferentes procesos, servidores o ubicaciones y se comunican mediante mecanismos de red.

Se usa principalmente para definir cómo se desplegarán las diferentes capas del sistema según necesidades de rendimiento, escalabilidad o distribución.

Capas Cerradas vs. Abiertas

Capas cerradas: una capa solo puede comunicarse directamente con la capa inmediatamente inferior. Para acceder a otra capa debe pasar por las capas intermedias.

Capas abiertas: una capa puede comunicarse directamente con capas inferiores sin tener que pasar obligatoriamente por todas las capas intermedias.

Diferencia principal: las capas cerradas ofrecen mayor control sobre las dependencias, mientras que las abiertas permiten mayor flexibilidad, pero pueden aumentar el acoplamiento si se utilizan excesivamente.

Características principales.

- **Separación de responsabilidades:** cada capa se encarga de una función concreta, como presentación, lógica de negocio o acceso a datos.
- **Niveles de abstracción:** las capas superiores trabajan con conceptos de mayor nivel, mientras que las inferiores manejan detalles más específicos de implementación.
- **Dependencia entre capas:** normalmente, una capa superior utiliza los servicios de una capa inferior, evitando que las responsabilidades se mezclen.
- **Encapsulamiento:** cada capa oculta sus detalles internos y expone únicamente los servicios necesarios para las demás capas.
- **Bajo acoplamiento:** busca reducir la dependencia directa entre componentes, facilitando modificaciones y mantenimiento.

- **Alta cohesión:** las funcionalidades relacionadas deben permanecer agrupadas dentro de la misma capa.
- **Comunicación controlada:** las capas se comunican mediante interfaces o servicios definidos, en lugar de acceder directamente a los detalles internos de otras capas.
- **Independencia de implementación:** una capa puede cambiar su implementación sin necesariamente modificar las demás. Por ejemplo, cambiar el sistema de base de datos sin modificar la interfaz de usuario.
- **Flexibilidad de despliegue:** las capas pueden ejecutarse en un mismo proceso o distribuirse entre diferentes procesos y servidores, dependiendo de las necesidades del sistema.
- **Escalabilidad y mantenibilidad:** la separación facilita modificar, probar y ampliar partes específicas del sistema sin afectar directamente a toda la aplicación.

Historia y evolución.

- Década de 1960 y 1970 - se inicia a aplicar principios de modularidad y encapsulación para mejorar la organización del software. En este contexto, los sistemas se comenzaron a dividir en módulos independientes, lo que llevó al concepto de capas.
- Años 80 y 90 - Con la popularización de las arquitecturas cliente-servidor (2-Tier), la separación entre las capas de presentación y lógica de negocio se hizo más evidente. Surgieron patrones arquitectónicos como Modelo-Vista-Controlador (MVC), que dividía las aplicaciones en capas específicas para la interfaz de usuario, lógica de negocio y acceso a datos.
- Años 2000, la Arquitectura en Capas se consolidó como un estándar de diseño en el desarrollo de software empresarial. Frameworks y tecnologías como Java EE, .NET y Spring adoptaron este enfoque, estableciendo una clara separación entre las capas de presentación, aplicación, negocio y persistencia.
- Hoy en día, aunque han surgido nuevas arquitecturas como Microservicios y Clean Architecture, la Arquitectura en Capas sigue siendo una opción sólida para proyectos donde la estructura y organización del código son esenciales para la mantenibilidad y escalabilidad del software.

Ventajas y desventajas.

Ventajas

- Separación de responsabilidades: Permite la separación de preocupaciones (SoC), ya que cada capa tiene una sola responsabilidad.
- Fácil de desarrollar: Este estilo arquitectónico es especialmente fácil de implementar, además de que es muy conocido y una gran mayoría de las aplicaciones la utilizan.
- Fácil de probar: Debido a que la aplicación está construida por capas, es posible ir probando de forma individual cada capa, lo que permite probar por separada cada capa.
- Fácil de mantener: Debido a que cada capa hace una tarea muy específica, es fácil detectar el origen de un bug para corregirlo, o simplemente se puede identificar dónde se debe aplicar un cambio.

- Seguridad: La separación de capas permite el aislamiento de los servidores en subredes diferentes, lo que hace más difícil realizar ataques.

Desventajas:

- Rendimiento: La comunicación por la red o internet es una de las tareas más tardadas de un sistema, incluso, en muchas ocasiones más tardado que el mismo procesamiento de los datos, por lo que el hecho de tener que comunicarnos de capa en capa genera un degrado significativo en el rendimiento.
- Escalabilidad: Las aplicaciones que implementan este patrón por lo general tienden a ser monolíticas, lo que hace que sean difíciles de escalar, aunque es posible replicar la aplicación completa en varios nodos, lo que provoca un escalado monolítico.
- Complejidad de despliegue: En este tipo de arquitecturas es necesario desplegar los componentes de abajo arriba, lo que crea una dependencia en el despliegue, además, si la aplicación tiende a lo monolítico, un pequeño cambio puede requerir el despliegue completo de la aplicación.
- Anclado a un Stack tecnológico: Si bien no es la regla, la realidad es que por lo general todas las capas de la aplicación son construidas con la misma tecnología, lo que hace amarres de la comunicación con tecnologías propietarias de la plataforma utilizada.
- Tolerancia a los fallos: Si una capa falla, todas las capas superiores comienzan a fallar en cascada.

Problemas comunes que se presentan y patrones.

Problemas Comunes:

- **Acoplamiento excesivo:** una capa depende demasiado de los detalles internos de otra, haciendo que los cambios sean difíciles de realizar.
- **Layer Skipping:** una capa accede directamente a otra capa inferior, ignorando las capas intermedias. Esto puede romper la separación de responsabilidades.
- **Capas demasiado grandes:** una capa termina concentrando demasiadas responsabilidades y se vuelve difícil de mantener.
- **Duplicación de lógica:** las mismas reglas o validaciones aparecen en diferentes capas.
- **Lógica de negocio en la presentación:** las reglas del negocio se implementan en controladores o interfaces en lugar de estar centralizadas en la capa correspondiente.
- **Lógica de negocio en persistencia:** los repositorios o componentes de acceso a datos comienzan a encargarse de reglas que deberían pertenecer al negocio.
- **Exceso de comunicación entre capas:** una operación sencilla debe atravesar muchas capas, generando código innecesario y, en arquitecturas distribuidas, mayor latencia.
- **Dependencias rígidas:** una modificación en una capa provoca cambios en varias capas superiores o inferiores.

Identificar patrones aplicables y cuando usarlos.

- **MVC (Model-View-Controller)**

- Separa la aplicación en tres responsabilidades principales: Model, encargado de los datos y la lógica; View, encargada de la presentación; y Controller, que gestiona las solicitudes y conecta ambas partes.
- Es útil principalmente en aplicaciones con interfaces web o gráficas donde se necesita separar la presentación de la lógica.
- **Service Layer**
 - Introduce una capa que concentra las operaciones y casos de uso de la aplicación. Esta capa recibe las solicitudes de las interfaces y coordina las operaciones necesarias entre otros componentes.
 - Es útil cuando varias interfaces necesitan utilizar las mismas operaciones de negocio.
- **Repository**
 - Proporciona una abstracción entre la lógica de negocio y la fuente de datos. La aplicación solicita operaciones al Repository sin necesitar conocer directamente cómo se almacenan o consultan los datos.
 - Es recomendable cuando se necesita separar la persistencia del resto del sistema.
- **Dependency Injection**
 - Las dependencias que necesita un componente se proporcionan desde el exterior en lugar de que el propio componente las cree. Esto permite reemplazar fácilmente una dependencia por otra.
 - Es útil para reducir el acoplamiento entre componentes y facilitar las pruebas.
- **Data Mapper / ORM**
 - Se encarga de relacionar los objetos utilizados por la aplicación con los datos almacenados en la base de datos. El mecanismo de mapeo se ocupa de transformar los datos entre ambos modelos.
 - Es útil cuando se necesita separar los objetos de la aplicación de los detalles de persistencia, especialmente al trabajar con bases de datos relacionales.
- **DAO (Data Access Object)**
 - Crea objetos especializados en realizar operaciones de acceso a datos, como consultar, insertar, actualizar o eliminar información. De esta manera, las demás capas no necesitan conocer los detalles de la persistencia.
 - Es útil cuando se busca separar el acceso a datos de la lógica de la aplicación.
- **Unit of Work**
 - Agrupa varias operaciones de persistencia dentro de una misma unidad de trabajo. Al finalizar, los cambios pueden confirmarse conjuntamente o descartarse si ocurre un error.
 - Es útil cuando una operación involucra múltiples modificaciones que deben gestionarse como un conjunto.
- **Active Record**
 - Combina los datos de una entidad con los métodos necesarios para trabajar directamente con su persistencia. La propia entidad puede encargarse de operaciones como guardar, actualizar o eliminar sus datos.
 - Es útil en aplicaciones donde las entidades necesitan gestionar directamente su propia persistencia.
- **Facade**

- Proporciona una interfaz sencilla para acceder a funcionalidades que internamente pueden involucrar varios componentes o procesos. La capa que utiliza la fachada no necesita conocer todos esos detalles internos.
- Es útil para simplificar la comunicación entre componentes y reducir dependencias innecesarias.
- **Adapter**
 - Actúa como intermediario entre componentes que utilizan interfaces diferentes. El Adapter transforma las solicitudes de una interfaz para que puedan ser entendidas por la otra.
 - Es útil cuando una capa necesita utilizar un componente cuya interfaz no coincide con la que se espera.
- **Proxy**
 - Introduce un intermediario entre un componente y quien intenta acceder a él. Este intermediario puede controlar, modificar o complementar las solicitudes antes de permitir el acceso.
 - Es útil para implementar funciones como validación, seguridad, control de acceso o registro de operaciones.
- **Retry**
 - Cuando una operación falla, el patrón permite volver a ejecutarla siguiendo determinadas condiciones y, normalmente, un número limitado de intentos.
 - Es útil cuando una capa depende de servicios externos o recursos que pueden presentar fallos temporales.
- **Timeout**
 - Establece un tiempo máximo que una operación puede permanecer esperando una respuesta. Si el límite se supera, la operación se considera fallida o se ejecuta un mecanismo alternativo.
 - Es útil para evitar que una capa quede bloqueada indefinidamente esperando a otro componente.
- **Health Checks**
 - Consiste en realizar comprobaciones para determinar si un componente o servicio está funcionando correctamente. Estas comprobaciones pueden verificar aspectos como disponibilidad o conexión con recursos necesarios.
 - Es útil para supervisar el estado de los componentes y detectar problemas en el sistema.

Casos de uso (cuándo usarlo y cuándo no).

Usar cuando:

- El sistema tiene responsabilidades claramente diferenciadas.
- Se necesita una arquitectura sencilla de comprender.
- El equipo conoce bien el modelo.
- Existe una aplicación empresarial tradicional.
- Se necesita separar UI, negocio y persistencia.
- El sistema no requiere una arquitectura altamente especializada.
- Se quiere una base sencilla para comenzar un proyecto.

No usar cuando:

- Sistemas con alta necesidad de escalabilidad independiente.

- Sistemas altamente orientados a eventos.
- Sistemas con dominio extremadamente complejo.
- Sistemas con requisitos extremos de rendimiento.

Casos de aplicación (ejemplos reales en la industria).

- **Sistemas bancarios:** se utiliza para separar las interfaces de los clientes, los servicios que gestionan operaciones bancarias, las reglas de negocio y los sistemas de almacenamiento de información.
- **Comercio electrónico:** Plataformas de compras pueden separar la interfaz web o móvil, la gestión de productos y pedidos, las reglas de precios y pagos, y la persistencia de los datos.
- **Sistemas empresariales (ERP/CRM):** aplicaciones para gestionar clientes, inventarios, ventas, empleados o facturación suelen organizarse en capas para mantener separadas las interfaces, procesos empresariales y bases de datos.
- **Sistemas universitarios:** plataformas de matrícula, gestión de estudiantes y calificaciones pueden dividirse en capas de presentación, aplicación, negocio y persistencia.
- **Aplicaciones web:** muchos sistemas web tradicionales utilizan una estructura similar separada en capas
- **Aplicaciones empresariales con Java o .NET:** frameworks como **Spring** y **ASP.NET** facilitan la implementación de aplicaciones organizadas mediante controladores, servicios y repositorios, siguiendo principios compatibles con una arquitectura por capas.

Frontend - Angular/JS

Definición clara (qué es y qué no es).

Es:

- Un framework de desarrollo web utilizado principalmente para crear aplicaciones web dinámicas y de una sola página.
- Un framework basado en componentes que proporciona herramientas para organizar la interfaz, gestionar datos, navegación, formularios y comunicación con servicios.
- Angular utiliza principalmente **TypeScript**, que se compila a JavaScript para ejecutarse en el navegador.

No es:

- Un lenguaje de programación. Angular es un framework - JavaScript y TypeScript si lo son.
- Un backend ni BD. Principalmente se utiliza para desarrollar la parte frontend de una aplicación.
- Un framework limitado únicamente a aplicaciones pequeñas o páginas estáticas.

Características principales.

- **Arquitectura basada en componentes:** divide la aplicación en componentes reutilizables que gestionan diferentes partes de la interfaz.
- **Data binding:** permite sincronizar los datos de la aplicación con los elementos de la interfaz.
- **Inyección de dependencias:** facilita la creación y reutilización de servicios y componentes.
- **Routing:** permite gestionar la navegación entre diferentes vistas de una aplicación web.
- **Formularios:** proporciona herramientas para crear y validar formularios.
- **Programación reactiva:** permite gestionar cambios y flujos de datos de forma dinámica.
- **Angular CLI:** proporciona herramientas para crear, configurar, desarrollar y compilar aplicaciones.
- **Aplicaciones de una sola página (SPA):** permite crear aplicaciones donde diferentes vistas pueden cargarse sin realizar una recarga completa de la página.
- **Modularidad:** permite organizar las funcionalidades de una aplicación en diferentes componentes y módulos para facilitar su mantenimiento.

Historia y evolución.

- Años 1990 - JavaScript: JavaScript surge como un lenguaje para agregar comportamiento e interactividad a las páginas web. Inicialmente se utilizaba principalmente mediante pequeños scripts dentro de documentos HTML.
- Años 2000 - Evolución del desarrollo web: JavaScript comienza a utilizarse para construir interfaces cada vez más dinámicas. Con técnicas como AJAX, las aplicaciones web pueden actualizar información sin recargar completamente la página, aumentando la complejidad del desarrollo frontend.
- 2009 - AngularJS: Google presenta AngularJS, un framework basado en JavaScript que busca facilitar la construcción de aplicaciones web dinámicas mediante características como data binding, inyección de dependencias, componentes y organización modular.
- 2016 - Angular: Google lanza Angular, una nueva generación del framework que reemplaza progresivamente a AngularJS. Angular mantiene JavaScript como base de ejecución y proporciona una estructura más completa para desarrollar aplicaciones web modernas.
- Actualidad: Angular continúa evolucionando como framework para aplicaciones web, mientras que JavaScript sigue siendo una de las tecnologías fundamentales del desarrollo frontend. Juntos permiten construir aplicaciones dinámicas, interactivas y de mayor complejidad.

Ventajas y desventajas.

Ventajas:

- Proporciona una estructura clara para proyectos grandes.

- Incluye muchas funcionalidades directamente en el framework.
- Facilita la reutilización mediante componentes y servicios.
- Cuenta con herramientas oficiales de desarrollo y soporte de Google.

Desventajas:

- Tiene una curva de aprendizaje relativamente alta.
- Puede resultar excesivo para proyectos pequeños.
- Tiene un ecosistema amplio que requiere mantener compatibilidades entre versiones.
- Su estructura puede generar más código y configuración que alternativas más simples.

Casos de uso (cuándo usarlo y cuándo no).

Usar cuando:

- Se desarrollan aplicaciones web medianas o grandes.
- Existen múltiples vistas, formularios y funcionalidades.
- Se necesita una estructura organizada para equipos de desarrollo.
- Se desarrollan sistemas empresariales, administrativos o dashboards.
- Se requiere una SPA con bastante lógica del lado del cliente.

No usar cuando:

- Solo se necesita una página estática sencilla.
- El proyecto tiene muy poca lógica e interacción.
- Se busca minimizar al máximo las dependencias y abstracciones.

Casos de aplicación (ejemplos reales en la industria).

Angular se utiliza principalmente en aplicaciones empresariales y plataformas web de gran tamaño.

Por ejemplo: **Google Cloud** y **Google Fonts**.

- Sistemas administrativos.
- Plataformas financieras.
- Sistemas educativos.
- Dashboards empresariales.
- Plataformas SaaS.
- Portales corporativos.
- Sistemas de gestión.

Relación entre el estilo y las tecnologías seleccionadas.

Angular puede utilizarse junto con una **arquitectura por capas**, principalmente para organizar la parte frontend.

Donde los componentes para Angular se encargan principalmente de la interfaz, mientras que los servicios del mismo pueden manejar lógica reutilizable y comunicación con el backend.

Que tan común es el stack designado (relación entre tecnologías).

El stack **Angular/JS** es un stack ampliamente utilizado para el desarrollo de aplicaciones web, especialmente en aplicaciones empresariales y proyectos de tamaño mediano o grande.

Angular funciona como el framework principal para estructurar y desarrollar la aplicación frontend, mientras que JS proporciona el lenguaje utilizado para ejecutar la lógica en el navegador.

Angular proporciona herramientas y estructuras para organizar la aplicación, mientras que JavaScript permite ejecutar el comportamiento y la lógica del lado del cliente.

Además, Angular puede utilizar otras tecnologías complementarias, como **HTML y CSS** para la estructura y presentación de la interfaz, y **Node.js y npm** como herramientas del entorno de desarrollo y administración de paquetes.

Backend - Fast API/Python

Definición clara (qué es y qué no es).

Es:

- Un framework web de Python orientado principalmente al desarrollo de APIs y servicios web.
- Una herramienta para construir backends que reciben solicitudes, procesan información y devuelven respuestas, normalmente mediante HTTP.
- Un framework basado en estándares como OpenAPI y JSON Schema, que permite generar documentación de la API automáticamente.

No es:

- Un lenguaje de programación. FastAPI es un framework y Python es el lenguaje.
- Una base de datos. FastAPI puede comunicarse con bases de datos, pero no almacena los datos por sí mismo.
- Un frontend. Su función principal es proporcionar servicios y APIs para que otras aplicaciones puedan consumirlos.

- Exclusivamente una herramienta para aplicaciones pequeñas; también puede utilizarse para APIs y servicios de producción de mayor escala.

Características principales.

- **Desarrollo de APIs:** permite crear endpoints para recibir y responder solicitudes HTTP.
- **Soporte asíncrono:** permite utilizar operaciones asíncronas mediante `async` y `await`.
- **Validación de datos:** utiliza las declaraciones de tipos y modelos para validar los datos recibidos.
- **Documentación automática:** genera documentación interactiva basada en OpenAPI.
- **JSON:** permite recibir y devolver datos en formatos compatibles con JSON.
- **Inyección de dependencias:** proporciona un sistema para gestionar dependencias entre componentes.
- **Integración con Python:** aprovecha las características y bibliotecas del ecosistema Python.

Historia y evolución.

- 2018 - creación de FastAPI: Sebastián Ramírez comienza el desarrollo de FastAPI buscando combinar características de diferentes herramientas existentes para crear APIs modernas en Python.
- Finales de la década de 2010: FastAPI incorpora conceptos como anotaciones de tipos de Python, programación asíncrona, OpenAPI y JSON Schema para simplificar el desarrollo y documentación de APIs.
- Años 2020: FastAPI aumenta su adopción dentro del ecosistema Python, especialmente para APIs, servicios y aplicaciones relacionadas con datos y aprendizaje automático.
- Actualidad: FastAPI continúa siendo un framework utilizado para desarrollar APIs de Python, con soporte para estándares como OpenAPI y JSON Schema y herramientas para documentación automática.

Ventajas y desventajas.

Ventajas:

- Permite desarrollar APIs rápidamente.
- Tiene validación y documentación integradas.
- Soporta programación asíncrona.
- Se integra bien con otras herramientas del ecosistema Python.
- Es adecuado para servicios que necesitan comunicarse mediante APIs.

Desventajas:

- Requiere conocimientos de Python y conceptos de desarrollo de APIs.

- Algunas aplicaciones pueden requerir configurar herramientas adicionales para autenticación, persistencia o tareas específicas.
- El desarrollo asíncrono puede aumentar la complejidad cuando no se conocen bien sus conceptos.

Casos de uso (cuándo usarlo y cuándo no).

Usar cuando:

- Se necesita desarrollar una API REST.
- Se requiere un backend para una aplicación web o móvil.
- Se necesita integrar modelos de aprendizaje automático mediante una API.
- Se requieren servicios que procesen datos y respondan a solicitudes.
- Se busca aprovechar el ecosistema Python.

No usar cuando:

- El proyecto solamente necesita una página web estática.
- No se necesita un backend.
- Se requiere principalmente una aplicación de escritorio sin comunicación mediante servicios web.

Casos de aplicación (ejemplos reales en la industria).

FastAPI se utiliza principalmente en aplicaciones y servicios de empresas

Por ejemplo: **Microsoft, Uber, Netflix y Cisco**

- APIs empresariales.
- Servicios de aprendizaje automático.
- Sistemas de procesamiento de datos.
- Backends para aplicaciones web.
- Microservicios.
- Servicios internos.

Relación entre el estilo y las tecnologías seleccionadas.

FastAPI puede funcionar como la **capa backend** dentro de una arquitectura por capas, donde se encarga principalmente de recibir las solicitudes provenientes del frontend, procesarlas mediante la lógica correspondiente y devolver los resultados.

Que tan común es el stack designado (relación entre tecnologías).

Python + FastAPI es un stack bastante utilizado dentro del desarrollo web con Python. Python proporciona el lenguaje, FastAPI proporciona la estructura para desarrollar el backend y la API utiliza HTTP y formatos como JSON para comunicarse con otras aplicaciones.

Base de Datos - MongoDB/Document

Definición clara (qué es y qué no es).

Es:

- MongoDB es una base de datos **NoSQL orientada a documentos**.
- Almacena la información en documentos BSON organizados dentro de colecciones.
- Su modelo permite representar los datos mediante estructuras similares a objetos y documentos JSON.

No es:

- Una base de datos relacional tradicional basada principalmente en tablas y filas.
- Un sistema que obligatoriamente requiere que todos los documentos tengan exactamente la misma estructura.
- Simplemente un archivo JSON. MongoDB almacena los documentos utilizando BSON, que amplía las posibilidades de JSON con tipos de datos adicionales.

Características principales.

- **Modelo orientado a documentos:** la información se almacena en documentos.
- **BSON:** utiliza una representación binaria de documentos similares a JSON.
- **Esquema flexible:** los documentos de una colección no necesitan tener exactamente los mismos campos.
- **Documentos embebidos:** permite almacenar información relacionada dentro de un mismo documento.
- **Referencias:** también permite relacionar documentos almacenados en diferentes colecciones.
- **Índices:** permite crear índices para mejorar el rendimiento de las consultas.
- **Escalabilidad:** está diseñado para trabajar con grandes cantidades de datos y cargas distribuidas.
- **Validación de esquemas:** aunque el modelo es flexible, pueden establecerse reglas para validar los documentos.

Historia y evolución.

- **2007 - Fundación de 10gen:** los fundadores de MongoDB comienzan el proyecto a partir de problemas encontrados al trabajar con grandes cantidades de datos en DoubleClick (empresa de gestión de anuncios en internet).
- **2009 - MongoDB:** el proyecto evoluciona hacia una base de datos orientada a documentos y comienza a ganar popularidad dentro del movimiento NoSQL.
- **Años 2010:** MongoDB incorpora mejoras en consultas, índices, replicación y escalabilidad, aumentando su uso en aplicaciones web y empresariales.

- **Años 2020:** MongoDB amplía su plataforma hacia servicios administrados en la nube mediante MongoDB Atlas y desarrolla funcionalidades para diferentes tipos de aplicaciones.
- **Actualidad:** MongoDB continúa utilizándose como base de datos documental para aplicaciones que requieren flexibilidad en la estructura de sus datos y diferentes opciones de escalabilidad.

Ventajas y desventajas.

Ventajas:

- Modelo de datos flexible.
- Los documentos pueden representar estructuras complejas.
- Permite almacenar documentos relacionados mediante estructuras embebidas.
- Facilita la evolución del modelo de datos.
- Cuenta con capacidades de consulta e indexación.
- Puede utilizarse en sistemas distribuidos.

Desventajas:

- La flexibilidad del esquema puede provocar inconsistencias si no se establecen reglas adecuadas.
- Algunos modelos de datos pueden requerir un diseño cuidadoso para evitar consultas complejas.
- No todos los problemas de datos son adecuados para un modelo documental.
- Las relaciones altamente complejas pueden ser más naturales en una base de datos relacional.

Casos de uso (cuándo usarlo y cuándo no).

Usar cuando:

- Los datos tienen estructuras variables.
- Se necesita evolucionar frecuentemente el modelo de datos.
- Los datos se pueden representar naturalmente como documentos.
- Se trabaja con catálogos de productos o contenido.
- Se desarrollan aplicaciones web, móviles o sistemas distribuidos.
- Se necesita almacenar grandes cantidades de documentos.

No usar cuando:

- El sistema depende principalmente de relaciones complejas entre muchas entidades.
- Se requieren numerosas operaciones basadas en joins relacionales.
- El modelo de datos es altamente estructurado, estable y encaja naturalmente con un modelo relacional.

Casos de aplicación (ejemplos reales en la industria).

MongoDB se utiliza en diferentes industrias

- Comercio electrónico.
- Servicios financieros.
- Telecomunicaciones.
- Retail.
- Salud.
- Entretenimiento.
- IoT.

Relación entre el estilo y las tecnologías seleccionadas.

MongoDB es una de las bases de datos documentales más conocidas y utilizadas dentro del ecosistema NoSQL. Su modelo documental también tiene una relación natural con JSON, aunque MongoDB utiliza **BSON** internamente. La aplicación normalmente intercambia información mediante JSON, mientras que MongoDB almacena los documentos utilizando BSON.

Que tan común es el stack designado (relación entre tecnologías).

MongoDB es una base de datos NoSQL bastante utilizada en aplicaciones web y sistemas que trabajan con datos semiestructurados. Su modelo de documentos permite almacenar información en estructuras JSON/BSON y se integra fácilmente con aplicaciones desarrolladas en diferentes lenguajes.

Dentro del stack, **MongoDB se encarga del almacenamiento**, mientras que **FastAPI/Python** maneja la lógica del backend y utiliza la base de datos para consultar y modificar información. **REST** define la forma en que el backend expone sus recursos, mientras que **JSON** facilita el intercambio de datos entre el frontend y el backend.

Integración - Rest, JSON, API

Definición clara (qué es y qué no es).

Es:

- **API (Application Programming Interface)** - Una interfaz que permite que diferentes aplicaciones o componentes se comuniquen mediante operaciones y reglas definidas.
- **REST (Representational State Transfer)** - Un estilo arquitectónico para diseñar sistemas distribuidos.
- **JSON (JavaScript Object Notation)** - Un formato de intercambio de datos ligero, textual e independiente del lenguaje.

No es:

- REST no es un lenguaje de programación.
- REST no es lo mismo que una API. REST es un estilo arquitectónico que puede utilizarse para diseñar APIs.
- JSON no es una API ni un protocolo de comunicación.
- Una API REST no necesariamente tiene que utilizar JSON, aunque JSON es uno de los formatos más utilizados para intercambiar información en este tipo de APIs.
- HTTP y REST tampoco son exactamente lo mismo. HTTP es un protocolo; REST es un estilo arquitectónico que puede utilizar HTTP como base de comunicación.

Características principales.

- **Separación cliente-servidor:** el cliente y el servidor mantienen responsabilidades separadas.
- **Comunicación mediante recursos:** los recursos se identifican mediante URI.
- **Métodos HTTP:** utiliza operaciones como GET, POST, PUT y DELETE para expresar diferentes acciones.
- **Stateless:** cada solicitud debe contener la información necesaria para procesarla, sin depender de un estado almacenado en el servidor entre solicitudes.
- **Representaciones:** los recursos pueden representarse mediante diferentes formatos, como JSON.
- **Interfaz uniforme:** busca establecer una forma consistente de interactuar con los recursos.
- **JSON:** proporciona una forma sencilla y legible de representar los datos intercambiados entre cliente y servidor.

Historia y evolución.

- **Años 1990 - Web y HTTP:** el crecimiento de la World Wide Web impulsa el desarrollo de sistemas distribuidos basados en protocolos de comunicación como HTTP.
- **2000 - REST:** Roy Fielding presenta en su tesis doctoral el estilo arquitectónico **Representational State Transfer (REST)** y explica los principios utilizados para diseñar la arquitectura de la Web.
- **Años 2000 - APIs web:** REST comienza a utilizarse ampliamente como enfoque para construir servicios web debido a su compatibilidad con HTTP y su modelo basado en recursos.
- **Años 2000 - JSON:** JSON comienza a ganar popularidad como formato de intercambio de datos debido a su simplicidad y compatibilidad con JavaScript y otros lenguajes.
- **Actualidad:** las APIs REST utilizan ampliamente HTTP y formatos como JSON para conectar aplicaciones frontend, backends, servicios externos y sistemas empresariales.

Ventajas y desventajas.

Ventajas:

- Es relativamente sencillo de implementar y consumir.
- Utiliza estándares ampliamente conocidos como HTTP.
- Permite separar frontend y backend.
- Facilita la integración entre tecnologías diferentes.
- JSON es ligero, legible y compatible con numerosos lenguajes.
- Permite que diferentes clientes consuman el mismo backend.

Desventajas:

- REST no define por sí mismo todos los detalles de una API concreta.
- El diseño incorrecto de endpoints puede producir APIs difíciles de mantener.
- Las respuestas JSON pueden contener información innecesaria si no se diseñan correctamente.
- Las aplicaciones pueden necesitar mecanismos adicionales para autenticación, autorización, versionamiento y control de errores.
- Para comunicaciones que requieren características diferentes, otros estilos o protocolos pueden ser más apropiados.

Casos de uso (cuándo usarlo y cuándo no).

Usar cuando:

- Un frontend necesita comunicarse con un backend.
- Una aplicación móvil necesita consumir servicios de un servidor.
- Diferentes sistemas necesitan intercambiar información.
- Se quiere exponer funcionalidades mediante una API.
- Se necesita integrar servicios externos.
- Se requiere una arquitectura donde cliente y servidor estén separados.

No usar cuando:

- La comunicación requiere principalmente tiempo real y actualización continua; pueden considerarse tecnologías como WebSockets.
- Se necesita una interfaz altamente especializada para consultas complejas; pueden considerarse otros enfoques como GraphQL.
- El sistema requiere comunicación interna con características específicas de rendimiento o serialización; pueden ser apropiados otros protocolos.

Casos de aplicación (ejemplos reales en la industria).

REST y JSON se utilizan ampliamente para conectar -

- Aplicaciones web.
- Aplicaciones móviles.

- Sistemas empresariales.
- Servicios de comercio electrónico.
- Plataformas educativas.
- Servicios financieros.
- Sistemas de integración entre empresas.

Dónde HTTP define los métodos y la semántica de las solicitudes y respuestas, mientras que JSON puede utilizarse como representación de los datos intercambiados.

Relación entre el estilo y las tecnologías seleccionadas.

REST funciona como el **mecanismo de integración** entre las diferentes partes del sistema, donde: Angular actúa como cliente, FastAPI proporciona los endpoints del backend y MongoDB almacena los datos. JSON sirve como formato de intercambio, lo que permite que las tecnologías sean relativamente independientes: el frontend no necesita conocer cómo funciona internamente FastAPI y FastAPI no necesita conocer cómo está construida la interfaz Angular.

La combinación **Angular/JavaScript + FastAPI/Python + REST + JSON + MongoDB** utiliza tecnologías y estándares ampliamente establecidos en el desarrollo web.

La relación entre ellas es la siguiente:

- **Angular/JavaScript:** construye la interfaz y actúa como cliente.
- **REST:** define el estilo de comunicación entre cliente y servidor.
- **HTTP:** transporta las solicitudes y respuestas.
- **JSON:** representa los datos intercambiados.
- **FastAPI/Python:** implementa el backend y los endpoints.
- **MongoDB:** almacena los datos como documentos BSON.

Que tan común es el stack designado (relación entre tecnologías).

El uso de **APIs REST** junto con **JSON** es muy común en aplicaciones web porque permite separar el frontend del backend y facilitar la comunicación entre diferentes componentes del sistema. REST define cómo se organizan y exponen los recursos, mientras que JSON es uno de los formatos utilizados para intercambiar los datos.

En el stack, **Angular/JS** consume la **API REST**, **FastAPI/Python** implementa los endpoints y procesa las solicitudes, **JSON** se utiliza para intercambiar la información y **MongoDB** almacena los datos que utiliza el backend.

Análisis Arquitectónico

Matriz de Atributos de Calidad vs. Estilo

Se usa el modelo de calidad del producto de la **ISO/IEC 25010:2023** .

Característica (ISO/IEC 25010:2023)	Subcaracterísticas más relevantes	¿Soporta o limita?	Justificación en Layered + Angular/FastAPI/MongoDB
Adecuación funcional (Functional suitability)	Compleitud funcional, corrección funcional, pertinencia funcional	Neutral	El estilo no garantiza que el sistema haga lo correcto. Lo que sí aporta es ubicar las reglas de negocio (validar stock y cliente activo al rentar un disco) en una sola capa, lo que facilita verificar que se cumplan.
Eficiencia de desempeño (Performance efficiency)	Comportamiento temporal, utilización de recursos, capacidad	Limita	Cada petición atraviesa Angular → controlador → servicio → repositorio → MongoDB y vuelve por el mismo camino. Ese recorrido añade latencia (comportamiento temporal) y hace que capas que solo delegan consuman recursos extra. Con pocos usuarios es tolerable; la capacidad (usuarios concurrentes) depende de escalar la API y MongoDB, no del estilo.
Compatibilidad (Compatibility)	Coexistencia, interoperabilidad	Soporta	El contrato REST/JSON entre Angular y FastAPI (documentado automáticamente con OpenAPI) permite que otros clientes consuman la misma API sin depender del frontend. Los tres contenedores (web, api, db) coexisten en la misma red Docker sin interferirse.

Capacidad de interacción (Interaction capability, antes usabilidad)	Protección contra errores de usuario, operabilidad, auto-descriptividad	Neutral	La experiencia de usuario depende del diseño de Angular, no de cómo se organizan las capas. El estilo ayuda de forma indirecta: al centralizar las validaciones en la capa de negocio se pueden devolver errores consistentes (409, 422) que la interfaz muestra al usuario.
Fiabilidad (Reliability)	Ausencia de fallos, disponibilidad, tolerancia a fallos, capacidad de recuperación	Limita (parcial)	Cada capa depende de la inferior, así que una caída de MongoDB se propaga hacia el servicio, la API y la interfaz (fallo en cascada). Se mitiga con manejo centralizado de excepciones, reintentos con tiempo límite y el replica set de MongoDB para la capacidad de recuperación.
Seguridad (Security)	Confidencialidad, integridad, no repudio, responsabilidad, autenticidad, resistencia	Soporta (parcial)	La separación por capas y por contenedores hace que solo la capa de persistencia hable con MongoDB (confidencialidad y menor superficie de ataque), y la validación de entradas en la API protege la integridad de los datos. Limitación: el estilo no da seguridad por sí solo; cada capa debe implementar sus controles y hay que evitar el <i>layer skipping</i> .

Mantenibilidad (Maintainability)	Modularidad, reusabilidad, analizabilidad, capacidad para ser modificado, capacidad para ser probado	Soporta	Es la mayor fortaleza del estilo: cada capa tiene una responsabilidad, por lo que un error se localiza rápido (analizabilidad), se puede cambiar el repositorio sin tocar el servicio (capacidad para ser modificado) y probar cada capa con dobles de prueba (capacidad para ser probado). Riesgo: capas demasiado grandes o lógica duplicada erosionan la modularidad.
Flexibilidad (Flexibility, antes portabilidad)	Adaptabilidad, escalabilidad, instalabilidad, reemplazabilidad	Mixto	La instalabilidad y la reemplazabilidad se soportan bien (Docker Compose levanta todo; el patrón Repository permite cambiar el motor de datos). La escalabilidad es limitada: el backend por capas corre como una sola aplicación, así que se escala replicando toda la API y no un componente aislado.
Protección (Safety)	Restricción operativa, identificación de riesgos, protección ante fallos, advertencia de peligro, integración segura	No aplica	Un sistema de renta de discos no controla procesos que puedan causar daño físico o económico grave, por lo que esta característica se documenta como no relevante para el caso.

Matriz de Principios vs. Estilo

Principio	¿Cómo lo cumple (o no) Layered Architecture?
SRP (Single Responsibility)	Cada capa tiene una única razón de cambio: presentación, lógica de negocio o acceso a datos. Es el principio que mejor encarna el estilo.
OCP (Open/Closed)	Con Repository + Dependency Injection en FastAPI, se pueden agregar nuevas implementaciones de persistencia sin modificar la capa de servicio. Se cumple si se usan interfaces/abstracciones; se rompe si el servicio importa directamente el driver de Mongo.

LSP (Liskov)	Aplica sobre todo en la capa de persistencia: cualquier implementación de un repositorio (Mongo hoy, otro motor mañana) debe poder sustituirse sin alterar el comportamiento esperado por la capa de servicio.
ISP (Interface Segregation)	Riesgo del estilo: es común crear un único "Service" o "Repository" grande que termina violando ISP. Se cumple solo si se definen interfaces específicas por caso de uso.
DIP (Dependency Inversion)	Es el principio más crítico en Layered: la capa de negocio (FastAPI) debe depender de una abstracción del repositorio, no de MongoDB directamente, para no violar la independencia de capas.
KISS	El estilo es, por diseño, uno de los más simples de entender y explicar (presentación → negocio → datos), lo cual favorece KISS frente a estilos como microservicios o event-driven.
DRY	Riesgo típico del estilo: duplicar validaciones entre el frontend Angular y el backend FastAPI. Se mitiga centralizando reglas de negocio en la capa de servicio y usando la misma capa como única fuente de verdad.
YAGNI	El estilo favorece YAGNI para un MVP: no exige de entrada infraestructura de mensajería, orquestación ni descubrimiento de servicios como sí lo requieren estilos distribuidos.
PoLA (Principio de mínima sorpresa)	Al ser un estilo ampliamente conocido, el comportamiento esperado (una petición baja capa por capa) es predecible para cualquier desarrollador que se una al equipo.
Ley de Demeter	Riesgo de "Layer Skipping": si un controlador FastAPI accede directamente a la colección de Mongo saltándose la capa de servicio, se viola Demeter y el aislamiento de capas.
STUPID (anti-principios)	El estilo es vulnerable a Tight Coupling (capas acopladas por accesos directos) y a Untestability (servicios con lógica y acceso a datos mezclados). Un buen uso de Repository + DI evita caer en STUPID.
Composición sobre herencia	No es inherente al estilo, pero se recomienda: los servicios de la capa de negocio deben <i>componerse</i> con repositorios inyectados en lugar de heredar de clases base de acceso a datos.

Matriz de Tácticas vs. Estilo y Stack (insumo para ADRs)

Característica ISO 25010:2023	Táctica	Aplicación en Layered + Angular/FastAPI/MongoDB	Fragmento de justificación tipo ADR
-------------------------------	---------	---	-------------------------------------

Mantenibilidad (capacidad para ser modificado)	Usar un intermediario (patrón Repository)	La capa de servicio en FastAPI nunca invoca el driver de MongoDB directamente; lo hace a través de una interfaz de repositorio.	Decisión: introducir Repository entre la capa de negocio y MongoDB. Contexto: el estilo exige bajo acoplamiento entre capas. Consecuencia: cambiar el motor de persistencia no impacta la capa de negocio.
Mantenibilidad (capacidad para ser probado)	Inyección de dependencias	FastAPI ofrece Depends(), que permite inyectar repositorios simulados en las pruebas.	Decisión: usar la inyección de dependencias nativa de FastAPI. Contexto: se necesita probar las reglas de renta sin levantar MongoDB. Consecuencia: pruebas unitarias rápidas con repositorios falsos.
Eficiencia de desempeño (comportamiento temporal)	Reducir saltos entre componentes	Endpoints y DTOs pensados para cada vista de Angular, de modo que una pantalla no encadene varias llamadas HTTP.	Decisión: diseñar endpoints orientados a la vista. Contexto: cada salto entre capas añade latencia. Consecuencia: menos llamadas desde el frontend a costa de DTOs adicionales.

Eficiencia de desempeño (utilización de recursos)	Indexar los datos consultados	Índices en MongoDB sobre los campos que más se buscan (título, artista, disponibilidad del disco).	Decisión: crear índices en la colección de discos. Contexto: la búsqueda en el catálogo es la consulta más frecuente. Consecuencia: lecturas más rápidas con un pequeño costo en escritura y almacenamiento.
Fiabilidad (tolerancia a fallos y capacidad de recuperación)	Reintentos, tiempo límite y verificación de salud	La capa de persistencia usa reintentos con <i>timeout</i> ante fallos temporales de MongoDB, traduce el error a un HTTP 503 controlado y la API expone un endpoint de <i>health check</i>. El replica set aporta recuperación.	Decisión: centralizar el manejo de fallos de MongoDB en la capa de persistencia. Contexto: evitar el fallo en cascada hacia Angular. Consecuencia: el frontend recibe errores homogéneos y puede degradarse con gracia.
Seguridad (integridad)	Validar las entradas	Modelos Pydantic en FastAPI validan tipos, rangos y campos obligatorios de cada petición antes de llegar a la capa de negocio.	Decisión: validar todo dato de entrada en el borde de la API. Contexto: la capa de presentación no es confiable. Consecuencia: datos inválidos se rechazan con 422 y no llegan a MongoDB.

Seguridad (confidencialidad y resistencia)	Limitar la exposición y aislar la capa de datos	En Docker Compose, MongoDB no publica su puerto al exterior: solo el contenedor api lo alcanza dentro de la red interna. Angular nunca conoce la cadena de conexión.	Decisión: mantener la base de datos en una red interna sin puerto expuesto. Contexto: el estilo separa la capa de datos de la de presentación. Consecuencia: menor superficie de ataque; el acceso a los datos solo pasa por la API.
Seguridad (responsabilidad)	Registrar las operaciones (auditoría)	Cada renta y devolución genera un registro con identificadores, fecha y resultado en la capa de negocio.	Decisión: registrar en un log estructurado cada operación de renta y devolución. Contexto: se debe poder rastrear qué pasó con cada disco. Consecuencia: trazabilidad para resolver disputas, con almacenamiento adicional de logs.
Compatibilidad (interoperabilidad)	Estandarizar el protocolo de comunicación	El contrato entre Angular y FastAPI usa REST con JSON:API para recursos, relaciones y errores, y OpenAPI documenta cada endpoint.	Decisión: adoptar REST con la convención JSON:API. Contexto: el stack requiere un protocolo de integración explícito. Consecuencia: cualquier cliente adicional consume la API sin documentación aparte.

Flexibilidad (instalabilidad)	Empaquetar en contenedores	Un contenedor por capa (web, api, db) orquestado con Docker Compose.	Decisión: desplegar con Docker Compose. Contexto: el enunciado exige contenedores y el estilo permite desplegar las capas por separado. Consecuencia: instalación reproducible con un solo comando.
Capacidad de interacción (protección contra errores de usuario)	Respuestas de error claras y consistentes	La capa de negocio responde 409 (sin stock) o 403 (cliente inactivo) y Angular traduce cada caso en un mensaje comprensible.	Decisión: definir un formato único de errores en la API. Contexto: el flujo de renta tiene rechazos previsibles. Consecuencia: el usuario entiende qué pasó y cómo corregirlo.

Matriz de Mercado Laboral vs. Estilo y Stack

Tecnología / rol	Popularidad / ranking actual	Salario mensual Colombia (COP)	Salario referencia global/US (anual)	Proyección / tendencia

JavaScript / Angular (Frontend)	JavaScript sigue siendo el lenguaje más usado en desarrollo web, con 66% de adopción entre desarrolladores según la encuesta 2025 de Stack Overflow, aunque en índices de búsqueda como TIOBE ocupa un puesto más bajo frente a Python.	Alrededor de \$3.9 a 4.0 millones/mes para "Desarrollador JavaScript" en Colombia (Computrabajo, 2026); el rango general de frontend web va de \$2.5 a \$7 millones/mes según nivel.	Salario global de desarrollador (mediana, todas las tecnologías) ronda US\$71.5K/año , con el 5% superior por encima de US\$216K (Stack Overflow 2025).	React, Angular y Node.js siguen entre las tecnologías más demandadas en Colombia según reportes de nearshoring (Talently, 2026); Angular mantiene demanda estable en proyectos empresariales grandes.
Python / FastAPI (Backend)	Python lidera tanto TIOBE (21.81%, feb. 2026 — su mayor ventaja histórica sobre el segundo lugar) como PYPL (31.17%, feb. 2026), y tuvo el mayor crecimiento de adopción interanual en la encuesta 2025 de Stack Overflow.	Un desarrollador Python en Colombia gana entre \$4 y \$9 millones/mes dependiendo de experiencia y sector (fintech, IA, datos); en portales de empleo como elemplo.com aparecen ofertas de \$6–7 millones/mes para roles backend Python.	En EE. UU., Python encabeza la demanda con más de 64.000 vacantes abiertas, por delante de Java y JavaScript (dato agregado 2025).	Python mantiene la proyección de crecimiento más fuerte entre lenguajes de backend por su uso en IA/datos, lo que indirectamente favorece frameworks como FastAPI.

MongoDB (Persistencia NoSQL)	MongoDB se mantiene como la base de datos NoSQL orientada a documentos más adoptada para aplicaciones cloud-native; reportes de mercado señalan un crecimiento de vacantes relacionadas con NoSQL cercano al 40% interanual (MoldStud, 2026).	No hay una serie robusta específica para Colombia; el salario de "Desarrollador backend" (perfil típico que usa Mongo en Colombia) va de \$3 a \$8 millones/mes (Computrabajo/UCC, 2026).	En EE. UU. el salario promedio de un "MongoDB Developer" ronda US\$84K–129K/año según la fuente (Salary.com vs. ZipRecruiter, 2026), y los ingenieros de software dentro de MongoDB Inc. reportan medianas bastante más altas (~US\$205K), lo cual refleja más el mercado interno de la compañía que el rol genérico.	Alta demanda sostenida por adopción de arquitecturas cloud-native y necesidad de flexibilidad de esquema; finanzas y salud se destacan como sectores que más contratan perfiles Mongo.
REST / JSON:API (Integración)	REST sigue siendo el protocolo de integración por defecto en la mayoría de stacks empresariales; no tiene un "ranking" propio pero aparece como habilidad co-requerida en la mayoría de vacantes backend (Python, Java, Node).	Se refleja indirectamente en los salarios de "Desarrollador backend" ya citados (\$3–8 millones/mes en Colombia), dado que REST es una habilidad transversal más que un rol independiente.	— (habilidad transversal, no tiene salario propio reportado por separado en las encuestas).	Se mantiene como estándar de facto; GraphQL y gRPC ganan terreno en nichos específicos pero no desplazan a REST en aplicaciones CRUD típicas como la de este taller.

Layered Architecture (como habilidad/estilo)	No se mide directamente en índices de lenguajes, pero sigue siendo el estilo por defecto enseñado y usado en aplicaciones empresariales tradicionales (Java EE, .NET, Spring, y ahora también FastAPI).	Se refleja en los mismos salarios de "Desarrollador backend/full stack" citados arriba, ya que es el estilo más común en ese tipo de vacantes.	—	Convive con microservicios y arquitecturas limpias; sigue siendo relevante para MVPs y sistemas medianos donde la simplicidad prima sobre la escalabilidad extrema.
---	---	--	---	---

Diseño

Entidades de negocio interrelacionadas

Entidad	Atributos clave	Relación
Cliente	id, nombre, email, telefono, estado (activo/inactivo)	1 Cliente → N Rentas
Disco	id, titulo, artista, genero, anio_lanzamiento, stock_total, stock_disponible	1 Disco → N Rentas
Renta	id, cliente_id (ref), disco_id (ref), fecha_renta, fecha_limite_devolucion, fecha_devolucion_real, estado (activa/devuelta/atrasada)	Entidad de asociación entre Cliente y Disco

HLD (High-Level Design)

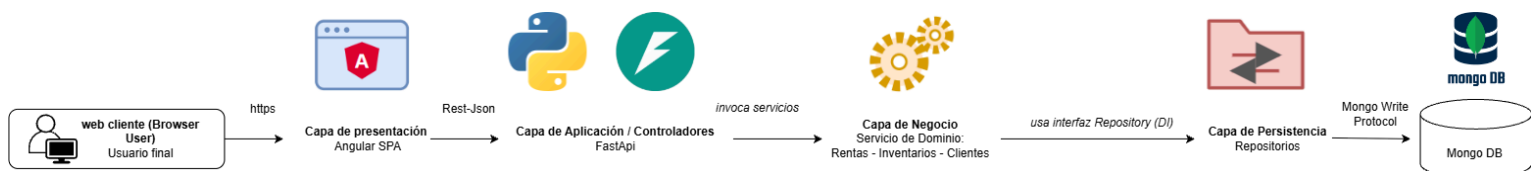


Diagrama de Contexto (C4 - Nivel 1)

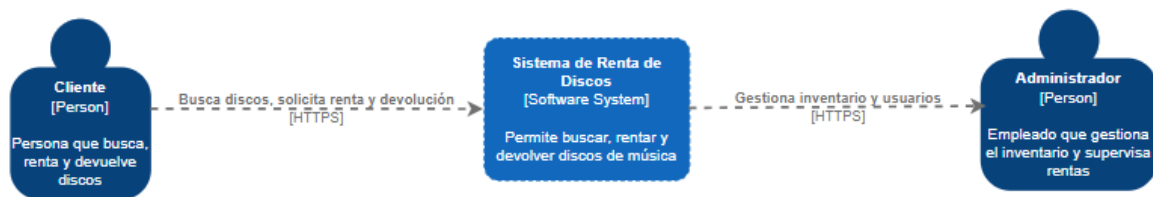


Diagrama de Contenedores (C4 - Nivel 2)

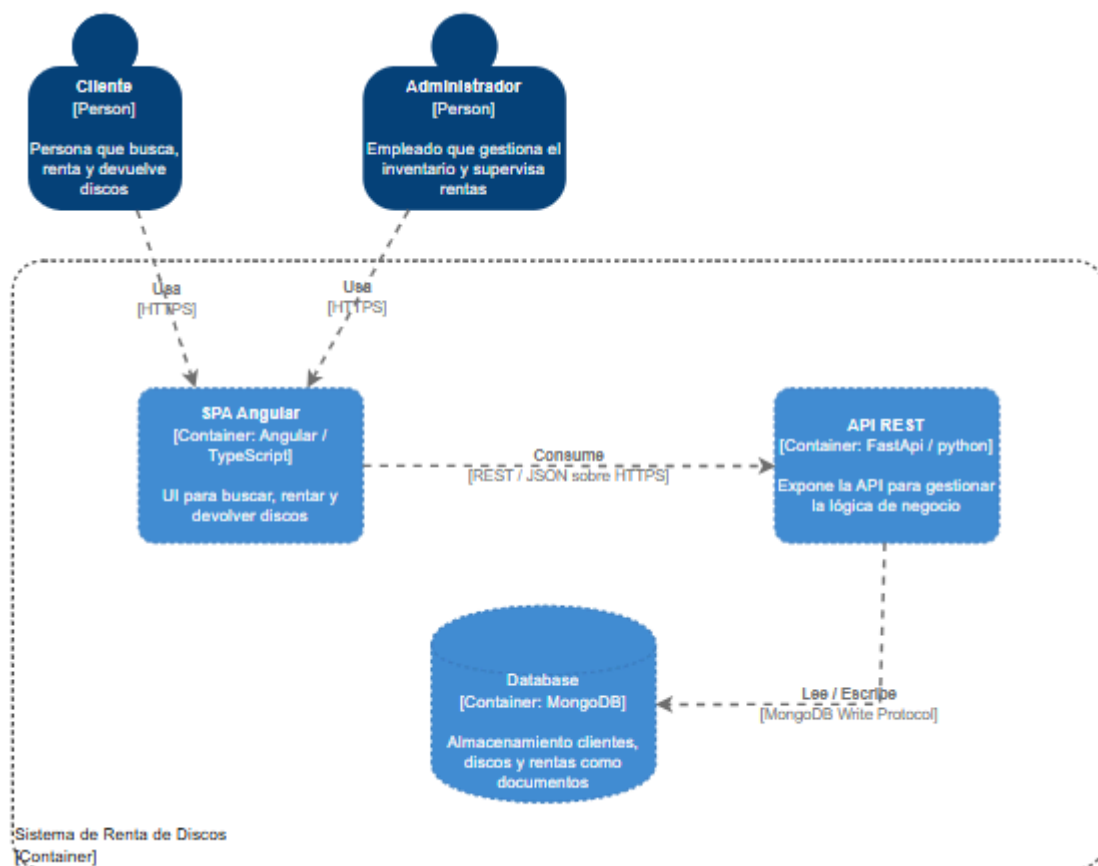


Diagrama Dinámico — Flujo principal: "Rentar un disco"

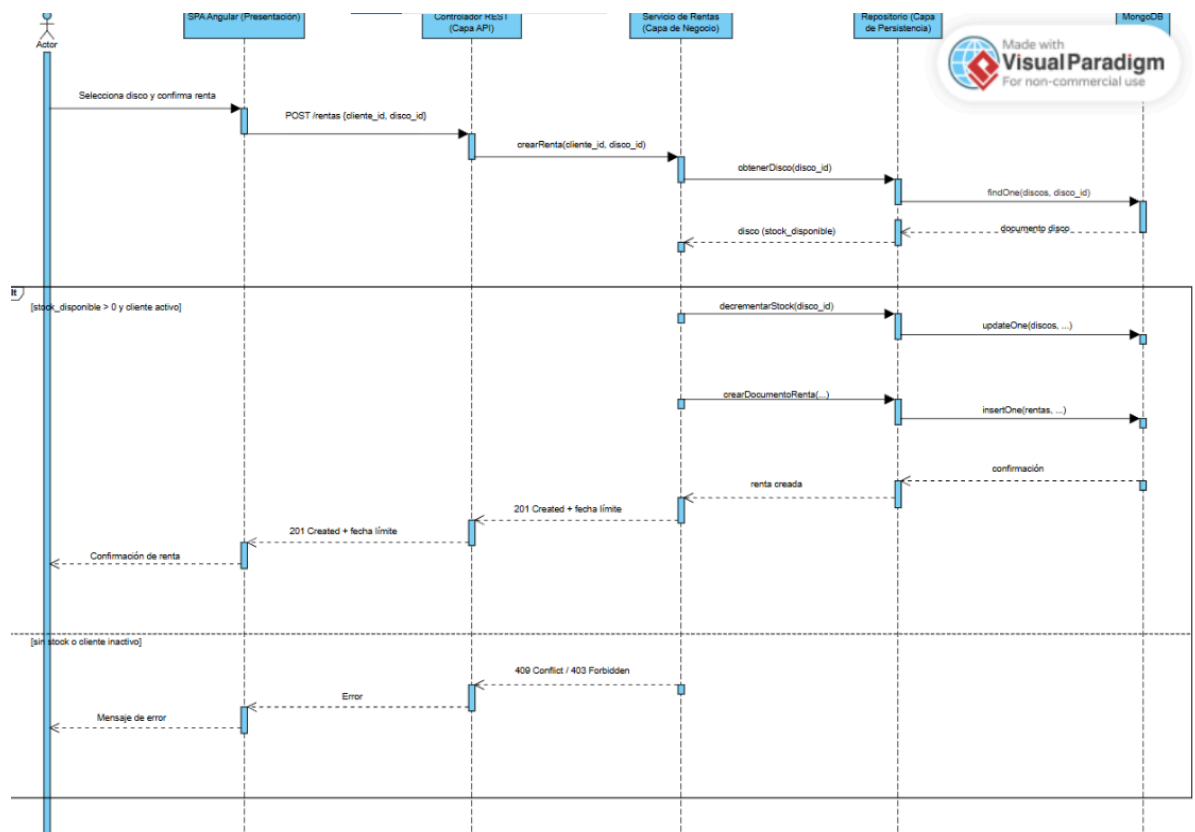
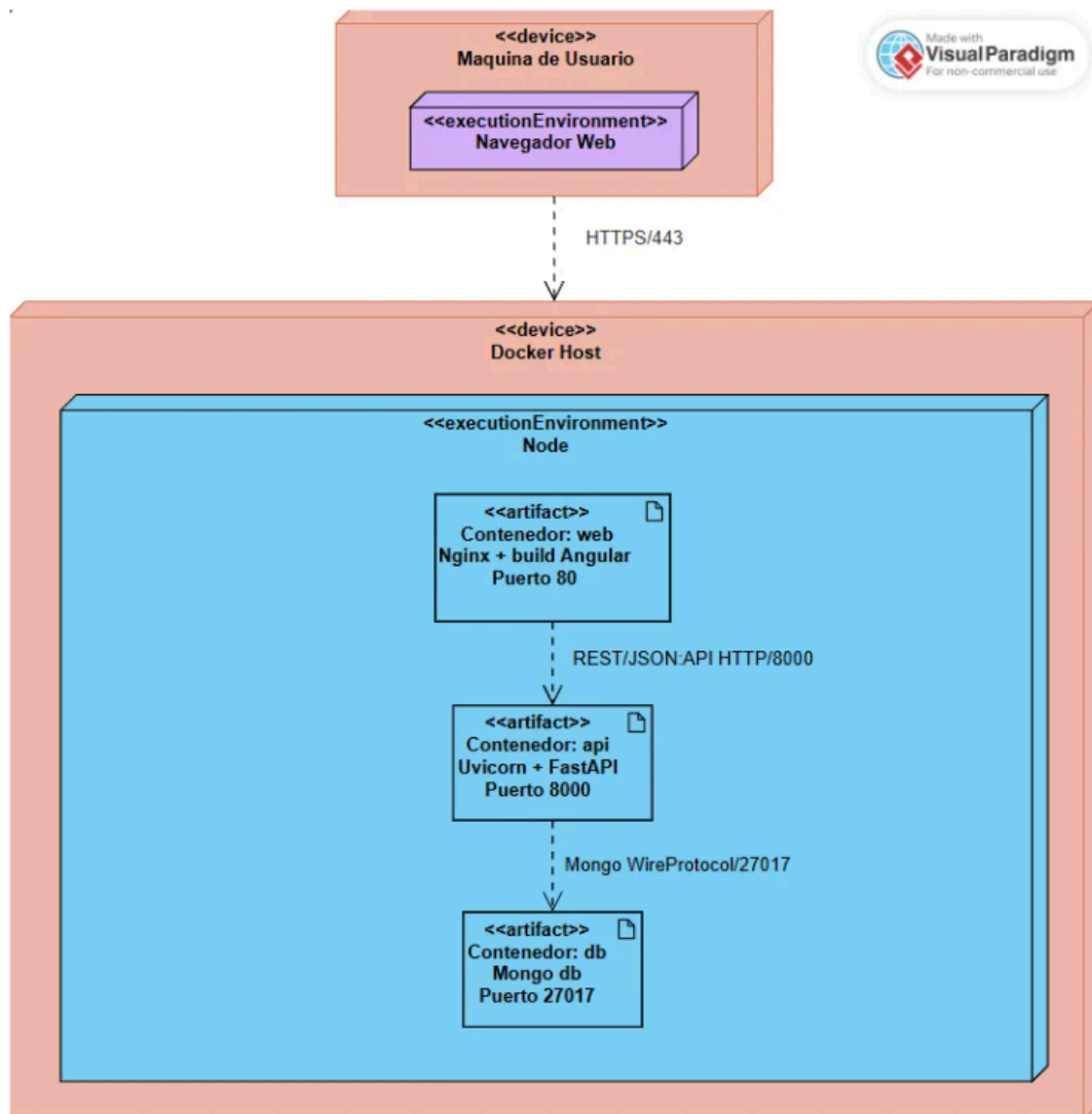


Diagrama de Despliegue



Lecciones Aprendidas

- Se comprende el funcionamiento de una arquitectura en capas, y que esta no define un número específico de estas, sirviendo como forma de estructurar proyectos de SW separando y definiendo las responsabilidades del programa en capas separadas..
- Se comprende el efecto de las capas separadas al momento de realizar cambios, pruebas y mantenimiento. Dónde patrones como Repository y Dependency Injection ayudan a evitar que una parte del sistema dependa demasiado de otra.
- Al analizar el stack, se comprende mejor el cómo cada tecnología cumple una función diferente dentro del sistema. Angular se encarga de la interfaz, FastAPI del backend, REST y JSON de la comunicación y MongoDB del almacenamiento de los datos.

- Finalmente, se reconoce como el elegir una tecnología no depende solamente de qué tan conocida o utilizada sea, sino de qué tan bien se adapta a las necesidades del sistema y a los problemas que se quieren resolver.

Fuentes

- <https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/capas>
- <https://www.mentorestech.com/resource-blog-content/arquitectura-en-capas-layered-architecture>
- https://www.cse.msu.edu/~cse870/Materials/Design/intro_softarch-Garlan-Shaw.pdf
- <https://angular.dev/overview>
- <https://angular.dev/essentials>
- <https://developer.mozilla.org/en-US/docs/Glossary/JavaScript>
- <https://docs.angularjs.org/api>
- <https://fastapi.tiangolo.com/>
- <https://fastapi.tiangolo.com/tutorial/first-steps/>
- <https://fastapi.tiangolo.com/python-types/>
- <https://fastapi.tiangolo.com/history-design-future/>
- <https://lp.jetbrains.com/python-developers-survey-2024/>
- <https://www.mongodb.com/es/docs/v8.0/core/document/>
- <https://www.mongodb.com/es/docs/manual/data-modeling/>
- <https://www.mongodb.com/company/our-story>
- <https://www.mongodb.com/resources/basics/json-and-bson>
- <https://www.mongodb.com/solutions/industries>
- <https://www.mongodb.com/resources/basics/databases/document-databases>
- <https://ics.uci.edu/~fielding/pubs/dissertation/abstract.htm>
- <https://martinfowler.com/eaCatalog/repository.html>
- <https://martinfowler.com/eaCatalog/dataMapper.html>
- <https://martinfowler.com/eaCatalog/unitOfWork.html>
- <https://www.martinfowler.com/eaCatalog/index.html>
- <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry>
- <https://learn.microsoft.com/en-us/azure/architecture/patterns/health-endpoint-monitoring>
- <https://learn.microsoft.com/en-us/azure/architecture/patterns>
- <https://www.rfc-editor.org/rfc/rfc9110.html>
- <https://www.rfc-editor.org/rfc/rfc8259.html>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference>
- <https://fastapi.tiangolo.com/>
- <https://survey.stackoverflow.co/2025/>
- <https://www.infoworld.com/article/4129615/python-is-slipping-in-popularity-tiobe.html>
- <https://blog.ucc.edu.co/cuanto-gana-programador-colombia>
- <https://co.computrabajo.com/salarios/desarrollador-javascript>
- <https://co.computrabajo.com/salarios/programadora>
- <https://www.empleo.com/co/ofertas-empleo/trabajo-desarrollador-python>
- <https://talently.tech/herramientas/colombia/salario>

- <https://www.ziprecruiter.com/Salaries/Mongodb-Developer-Salary>
- <https://www.geeksforgeeks.org/mongodb/mongodb-engineer-salary-and-skills-required/>